PreAct: Computer-Using Agents that Get Faster on Repeated Tasks

Bojie Li
Pine AI

Abstract

A computer-using agent (CUA) that adds a contact named "Jordan Chen" to the address book on Monday re-derives the entire solution from scratch on Tuesday: it pays the same few seconds of screenshot-reading and reasoning *per step* for a UI traversal it has already performed. We present **PreAct**, a harness that makes a CUA *faster every time it repeats a task*. The first time the agent succeeds, PreAct compiles the live trace into a *state-machine program* that is at once the record of what happened and a directly executable artifact; on the next invocation the harness simply walks that graph—checking each state's predicate against the live screen and firing each transition's action—and falls back to the full agent only when a check fails. **The artifact is the runtime**: what the agent remembers is what it can directly execute, and warm runs are an order of magnitude cheaper (Figure 1).

This promise has a catch: an agent that blindly trusts its own compiled programs gets faster *and wrong*. A program can replay to completion and still leave the task unsolved—the contact silently not saved. PreAct's load-bearing mechanism is a *verify-before-store gate* that re-runs each freshly compiled program against a clean environment and keeps it only if an independent evaluator confirms it worked. A 2×2 ablation (cold/warm $\times$ gate on/off) on three structurally different platforms—AndroidWorld, OSWorld, and WebArena—shows the gate is precisely what makes the speedup *monotonic*: with the gate on, repeated runs improve; with it off, they regress as the corpus fills with programs that run but do not work (§4.3, Figure 9). A second mechanism—a cache-miss fallback to the full agent—closes the remaining gap to a strong record-and-replay baseline, MuscleMem [5] (§4.4). Equally informative are the *negative* findings: the system's prompts and runtime guardrails turn out not to be load-bearing, and a plain embedding retriever matches the LLM-based program selector. The harness—two cooperating mechanisms over a verified, self-extending corpus—is the contribution.

1 Introduction

1.1 The same task, every day, from scratch

Consider a concrete task: *add a new contact named "Jordan Chen" with a given phone number*. A computer-using agent built on the Observation–Reasoning–Action paradigm [9, 1] solves it by reading a screenshot, reasoning about what to tap, acting, and repeating—a half-dozen rounds of vision-and-language inference. Run the *same* task tomorrow and the agent does all of it again: the screen is the same, the taps are the same, but the agent has no memory it can execute, so it pays the full per-step inference cost a second time. We call this the **rerun crisis**:

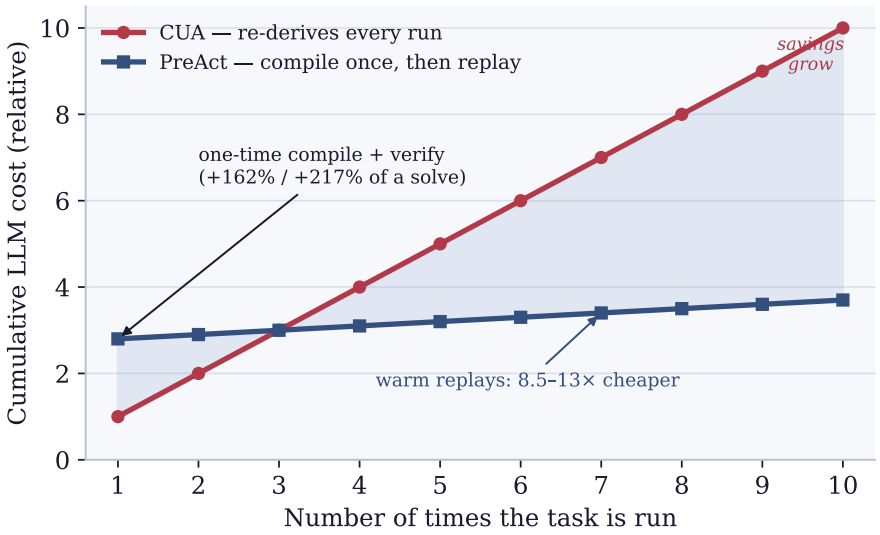

Figure 1. The rerun crisis, and PreAct's promise. A standard CUA re-derives a familiar task on every run, so its cumulative cost grows linearly (red). PreAct pays a one-time premium to compile and verify the first success (the bump at run 1), then replays the compiled program on every subsequent run at 8.5–13× lower cost (blue). The more often a task recurs, the more the harness saves. Cost is normalised to one full agent solve; the speedup and compile-overhead annotations are measured (§4.2, §4.2).

a user with M daily tasks and N steps each pays $O(M \times N)$ language-model calls every day, most of them re-deriving traversals the agent has already discovered.

Why prior work does not close the gap. Three lines of work attack reuse, each stopping short of executing what it stores. *Skill systems* (e.g., SAGE [4]) save agent behaviours as parameterised skills, but the skills still run inside an LLM-bound loop, so most of the per-step cost remains. *Compilation systems* (Compiled-AI [7], ActionEngine [10]) emit deterministic code, but target narrow business logic or generate flat Python from a state machine built by untargeted app crawling rather than from a real task success. *Record-and-replay systems* (Muscle-Mem [5], AgentRR [3], Workflow-Use [2]) cache a linear action sequence and re-run the agent on a cache miss, but the stored trace has no per-state verification, no branching, and no way to improve itself.

1.2 PreAct: the artifact is the runtime

PreAct makes one commitment the others do not: **the artifact the agent stores is the program it later runs**. The first time the agent solves "add Jordan Chen," PreAct compiles the live trace into a *state-machine program*—a small graph whose states carry verification predicates ("the contact form is open") and whose transitions carry actions ("tap Create new contact"). On the next invocation the harness retrieves that graph and *walks* it: at each state it checks the predicate against the live screen, and at each transition it fires the action. No flat-script generation step (cf. ActionEngine), no LLM in the execution loop (cf. skill systems), no blind linear replay (cf. Muscle-Mem). If any predicate fails, the harness falls back to the full agent, re-compiles, and—only if the result passes a verification gate—extends or replaces the stored program. The corpus thus *self-extends*: the harness code is fixed; the executable corpus is the part that grows, and it grows only through verified compile cycles (Figure 4).

Table 1. PreAct vs. related approaches.

System	Memory representation	Replay fidelity	Fallback path	Self-extension
Muscle-Mem	Linear tool calls	Direct	Agent on cache miss	Append cache only
AgentRR	Multi-level experiences	Indirect	LLM	Append only
Workflow-Use	Linear scripts	Direct	Agent	Append only
ActionEngine	State machine via crawl	Flat-Python generated	None	Re-crawl from scratch
PreAct	**State machine**	**The SM IS the runtime**	**CUA + recompile under verify-gate**	**Verified corpus growth + dedup-replacement**

1.3 Contributions

1. **An "artifact-is-runtime" harness** (§3). Each successful run compiles into a state-machine program that the harness executes directly by walking the graph—not a flat script, and not a trace replayed inside an LLM loop. The corpus self-extends through verified compile cycles.
2. **A verify-before-store gate, shown to be load-bearing** (§4.3). A program enters the corpus only if, re-run from a clean state, an independent evaluator agrees it solved the task. A cold/warm $\times$ gate-on/off ablation on three structurally different platforms (AndroidWorld, OSWorld, WebArena) shows the gate is what makes repeated-task performance improve rather than decay (Figure 9).
3. **A cache-miss fallback that closes the gap to record-and-replay** (§4.4). When the gate rejects every candidate for a task, the harness runs the full agent rather than returning nothing—matching the strong Muscle-Mem baseline on WebArena while keeping the gate's benefit elsewhere.
4. **Monotonic, cross-model refinement** (§4.2). Repeated runs improve across three seeds and reproduce under a second model, with an identical stable failure set that we trace to the benchmark rather than the method.
5. **Negative results that sharpen the claim** (§4.6). The system's prompt content and runtime guardrails are *not* load-bearing, and a plain embedding retriever matches the LLM-based selector. **The harness is the contribution.**

2 Related Work

We compare PreAct against four classes of prior work: pure CUA baselines, skill-based systems, compilation-based systems, and concurrent record-replay systems. Table 1 summarizes the comparison along five dimensions.

The artifact is the runtime. The commitment that distinguishes PreAct is the second column (*the SM is the runtime*) combined with the fourth (*recompile under verify-gate*). ActionEngine builds a state machine but emits flat Python from it; the state machine is consumed at compile time and the runtime is the generated script. PreAct executes the state machine directly: each state's verification predicate is evaluated against the live UI, and each transition is dispatched

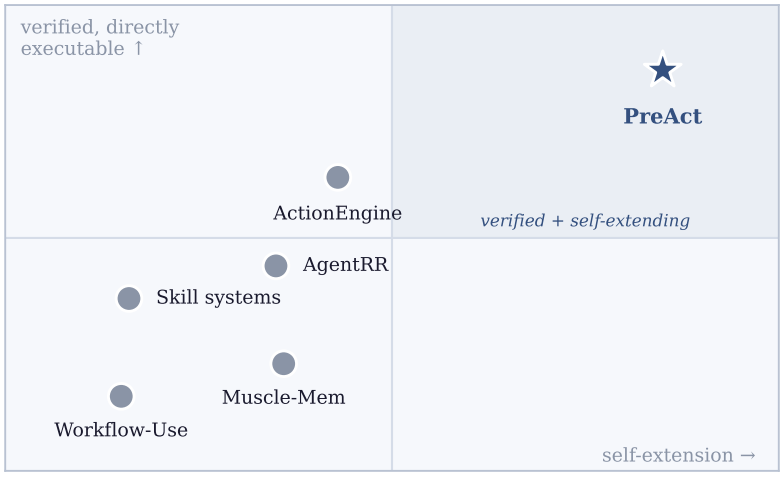

Figure 2. Where PreAct sits. Prior systems either store directly-runnable artifacts *or* refine what they store, but not both, and none verifies a stored artifact against an independent evaluator. PreAct occupies the upper-right: a verified, self-extending corpus of directly-executable programs. Axes are qualitative.

to the underlying action backend (XPath click, JSON action, etc.). This direct execution enables (a) per-state verification and graceful fallback, (b) in-place patching when a transition fails, and (c) the verify-before-store gate (which we will argue is empirically necessary for monotonicity).

What grows with experience. The deeper distinction concerns *what grows* as the agent gains experience. In Muscle-Mem, AgentRR, and Workflow-Use, the cache or experience store is append-only: every captured trajectory adds a new entry without affecting the existing ones. In ActionEngine, the state machine is built once via crawling and the flat Python is regenerated wholesale on each rebuild; there is no notion of incremental refinement. PreAct's compile-extend-replace loop, by contrast, allows the corpus to *mutate*: a fresh compile with the same dedup signature replaces an older program if it passes the verify-gate. The corpus does not just accumulate—it refines its representation of repeated tasks as the agent encounters them across different parameter values, environments, and sessions. This is the property we mean by "self-extending executable code corpus" (§1): not an append-only cache, but a mutable, verified, directly-executable code library.

3 System Architecture

3.1 What the agent stores: a state-machine program

We start from the artifact, because in PreAct the artifact *is* the runtime. Listing 1 is an *actual* program from the corpus—the one compiled the first time the agent added a contact (the trace happened to be for "Emilia Gonzalez"). It is a small graph, drawn in Figure 3: each *state* carries a verification predicate (an accessibility-tree pattern that must hold on the live screen for the agent to believe it is in that state) and each *transition* carries an action. The concrete values typed during the trace are lifted to *parameters*—here `first_name`, `last_name`, `phone_number`, listed in the metadata—so this single program serves every future "add a contact" request, including "add Jordan Chen," by binding the parameters to new values.

```
{
```

```
  "metadata": {
    "task_description": "Create a new contact for Emilia Gonzalez. ...",
    "application_context": "com.google.android.contacts",
    "parameters": ["first_name", "last_name", "phone_number"]
  },
  "states": [
    { "id": "contacts_app_open",   "verification": {"type":"expect_element", "xpath":"resource_id=....:
     id/floating_action_button"} },
    { "id": "create_contact_form", "verification": {"type":"expect_element", "xpath":"class=android.
     widget.EditText&&hint=First name"} },
    { "id": "first_name_entered",  "verification": {"type":"expect_element", "xpath":"class=android.
     widget.EditText&&hint=Last name"} },
    { "id": "last_name_entered",   "verification": {"type":"expect_element", "xpath":"class=android.
     widget.EditText&&hint=Phone"} },
    { "id": "phone_entered",       "verification": {"type":"expect_element", "xpath":"resource_id=
     android:id/text1&&text=Mobile"} },
    { "id": "phone_type_selected", "verification": {"type":"expect_element", "xpath":"...:id/
     toolbar_button&&text=Save"} },
    { "id": "contact_saved",       "verification": {"type":"terminal_state"} }
  ],
  "transitions": [
    { "from_state":"contacts_app_open",   "to_state":"create_contact_form",
      "action": {"type":"action_click", "target":"resource_id=....:id/floating_action_button"} },
    { "from_state":"create_contact_form", "to_state":"first_name_entered",
      "action": {"type":"action_type", "parameter_name":"first_name",   "target":"...hint=First name"}
     },
    { "from_state":"first_name_entered",  "to_state":"last_name_entered",
      "action": {"type":"action_type", "parameter_name":"last_name",    "target":"...hint=Last name"} },

    { "from_state":"last_name_entered",   "to_state":"phone_entered",
      "action": {"type":"action_type", "parameter_name":"phone_number", "target":"...hint=Phone"} },
    { "from_state":"phone_entered",       "to_state":"phone_type_selected",
      "action": {"type":"action_click", "target":"...text=Mobile"} },
    { "from_state":"phone_type_selected", "to_state":"contact_saved",
      "action": {"type":"action_click", "target":"...text=Save"} }
  ]
}
```

Listing 1. The actual compiled program for "add a contact," taken from the corpus (program `ab4390a9`). It is stored as JSON; here, schema-default `null` fields, per-state natural-language descriptions, and long resource-id package prefixes (…) are elided for space—the verbatim record is in Appendix A. Figure 3 draws the same program.

Formally a program is a tuple $P = (S, T, M, V)$: states S (id, description, verification predicate), transitions T (each (s_i, s_j, a) firing action a to move from s_i to s_j), metadata M (task description, app context, parameter schema, and a *dedup signature* that identifies which task family the program serves), and data-extraction predicates V for question-answering tasks (e.g. `inspect_text`). The full action vocabulary is in Appendix B. This graph form buys three things a flat script cannot: **(1)** per-state verification, so the agent knows immediately when an action did not have the intended effect; **(2)** branching, since one state can have several outgoing transitions chosen by predicate (a delete-confirmation dialog that appears only on some app versions); and **(3)** in-place extension, since a new state and transition can be spliced in without regenerating the whole program.

3.2 The loop: select, replay, fall back, verify

Figure 4 shows how the harness uses such programs. For a goal T:

Figure 3. The compiled "add a contact" program of Listing 1 drawn as a state machine: each box is a state holding a verification predicate ($\models$) that must match the live screen; each arrow is a transition holding an action. The harness executes this graph directly—it is not regenerated into a flat script.

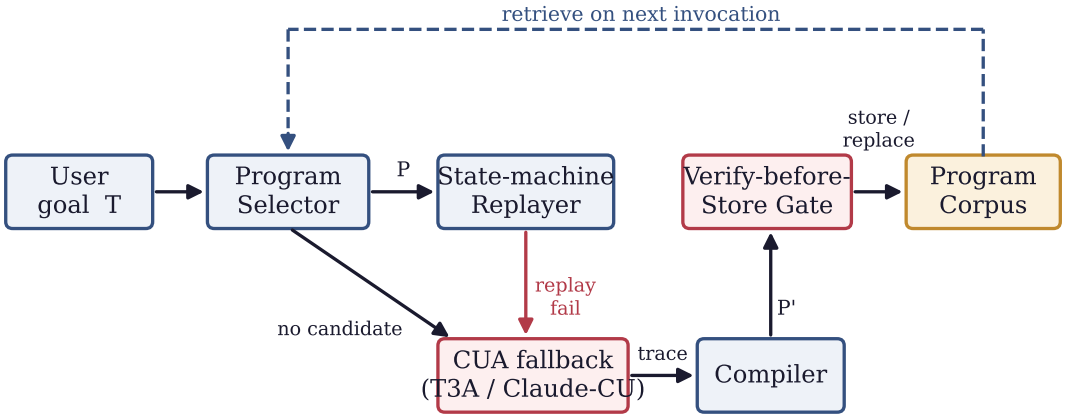

Figure 4. The PreAct harness is a verified compile–extend–replace loop. A goal is routed to the *Program Selector*; a retrieved program is run by the *Replayer*, which walks the graph and checks each predicate against the live screen. On any miss the harness falls back to the full agent (*CUA*), whose trace is compiled into a new program P'. P' enters the corpus only through the *Verify-before-Store Gate*. The corpus is the only growing structure; the harness code is fixed.

1. **Select.** The Program Selector queries the corpus with T and returns either a candidate program P or "no candidate."
2. **Replay.** If P exists, the Replayer walks its graph—checking each state's predicate against the live screen and firing each transition's action. "Add Jordan Chen" on Tuesday is solved here, in seconds, with no language-model call.

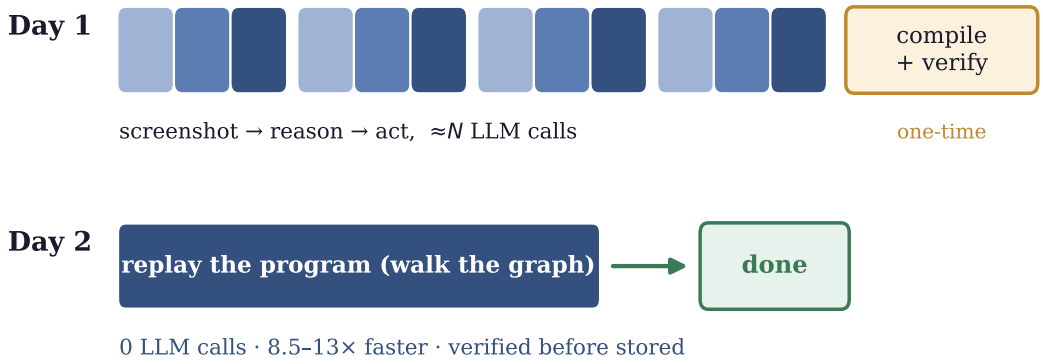

Figure 5. Where the cost goes. The first time PreAct sees a task it runs the full agent (top: $\approx N$ rounds of observe–reason–act) and pays a one-time compile-and-verify premium. Every later run replays the stored program (bottom): no language-model calls, an order of magnitude faster, and already verified before it was stored.

3. **Fall back.** If a predicate is false or an action errors, the harness hands control to the full agent (CUA), which continues from the live screen and records a fresh trace.
4. **Verify and store.** On success the trace is compiled into a new program P'; the verify-before-store gate (§3.3) re-runs P' from a clean state and stores it—replacing any program with the same dedup signature—only if it independently passes.

The static harness code defines this loop; the corpus is what grows, and step 4's gate is what lets it grow *monotonically* rather than accumulate programs that run but do not work. Algorithm 1 makes the loop precise.

Three properties of Algorithm 1 are worth highlighting. First, **the harness's only side effect on C is line 16's UPSERT call, gated by line 15's double predicate**: programs enter the corpus only after independent re-validation. Second, **the corpus grows monotonically in expectation**: each UPSERT call adds a program that has independently re-passed both replay and evaluation, so the corpus quality cannot decrease across UPSERT events. Third, **the corpus mutates via UPSERT, not just append**: a fresh program with the same dedup signature replaces an older one, allowing the corpus to refine its representation of repeated tasks rather than just accumulate variants.

3.3 The verify-before-store gate

The gate is the mechanism that separates PreAct from a blind cache. When the agent succeeds and the trace is compiled into a candidate P', the harness does *not* trust it. It resets the environment to the task's clean initial state, replays P' there, and asks the benchmark's own evaluator whether the task is solved. P' is stored only if *both* hold: the replay ran to its terminal state without error (`replay_ok`) *and* the evaluator scores it a pass (`score` ≥ 1.0).

Both halves earn their place. A single `score` ≥ 1.0 check lets through programs whose actions silently no-op (a malformed action the backend swallows) while the environment happens to still hold passing state from an earlier step—so we additionally require the replay

Algorithm 1 PreAct verified compile-extend-replace loop

Require: Task goal T, environment env, program corpus C, replay-coverage threshold τ (default 0.5)

$P \leftarrow \textsc{Selector}(T, C)$ ▷ LLM-agentic; may return $\perp$
$r \leftarrow \textsc{None}$
if $P \neq \perp$ **then**
  $r \leftarrow \textsc{Replay}(P, env)$ ▷ walk graph; verify each state predicate
  if $r.success \wedge r.cov > \tau$ **then**
    return r ▷ warm path: trusted replay
  end if
end if
$r \leftarrow \text{CUA}(T, env, r)$ ▷ hybrid: when $r \neq \textsc{None}$, CUA continues from the partial-replay state
if $\neg r.success$ **then return** r
end if
$P' \leftarrow \textsc{Compile}(r.trace)$ ▷ LLM-driven trace → state-machine
$env' \leftarrow \textsc{Reset}(env, T)$ ▷ independent re-evaluation environment
$r' \leftarrow \textsc{Replay}(P', env')$
$score' \leftarrow \textsc{Evaluate}(env', T)$
if $r'.success \wedge score' \geq 1.0$ **then** ▷ double gate
  $C \leftarrow \textsc{Upsert}(C, P')$ ▷ insert by dedup signature; replace if collision
end if
return r

to report its own success. The converse and more important failure mode is `replay_ok` *with* `score` $= 0$: the program executes every action to completion (100% coverage) yet the end state does not satisfy the evaluator—the contact form was filled and "Save" was tapped, but a stale field meant no contact was written. We call this the **cov=100%/score=0 lossy replay**, and §4.3 shows it is exactly what the gate must catch.

3.4 Selector and fallback

Two supporting components complete the loop. The **Program Selector** decides which stored program (if any) applies to a new goal; by default it is an LLM that reads the goal and each program's description and parameter schema and chooses via a tool call, but §6 shows a plain embedding retriever does just as well—the selector's implementation is not load-bearing. The **CUA fallback** is the full agent, invoked whenever the selector finds no candidate or a replay fails; we use AndroidWorld's T3A agent on mobile and Anthropic's Computer-Use API on desktop and web. Notably, the production fallback runs the *verbatim* upstream agent prompt with no PreAct-specific additions (§4.6)—the harness, not the prompt, is what makes the system work.

4 Empirical Validation

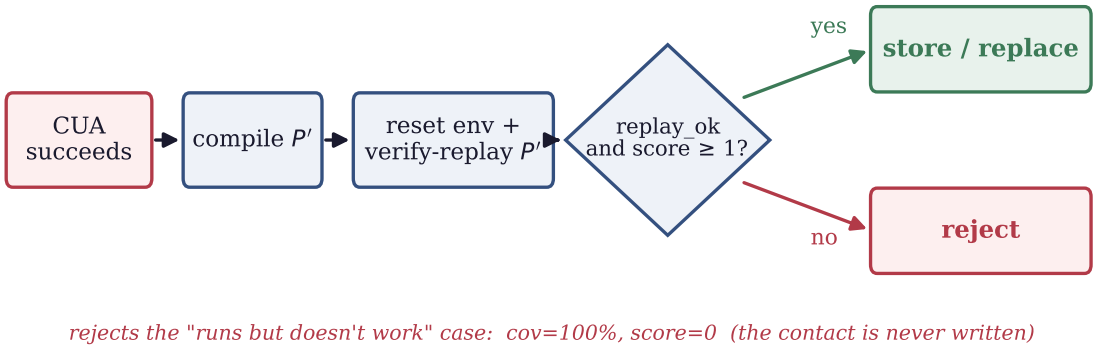

Figure 6. The verify-before-store gate. A freshly compiled program is re-run from a clean state and re-scored by an independent evaluator; it enters the corpus only if it both replays cleanly *and* passes. This is what rejects the "runs but doesn't work" programs—the ones that replay to full coverage yet leave the task unsolved.

4.1 Setup

Benchmarks. We evaluate on (a) AndroidWorld [6] "official-15" subset (15 tasks across 8 application domains; the curated subset used by the upstream T3A baseline), (b) OSWorld [8] "test_tiny" subset (6 tasks across Chrome, LibreOffice Calc, and LibreOffice Writer), and (c) a 12-task shopping_admin subset of WebArena [11] (string-match answer-extraction tasks on a Magento e-commerce admin panel).

Models. For Android CUA, we use Gemini 3 Flash via the multi-provider port; this is roughly 10× cheaper than Claude Sonnet 4.6 at equivalent SR (we confirm this with cross-model replication in §4.2). For OSWorld and WebArena CUA, we use Claude Sonnet 4.6 via Anthropic's Computer-Use API. For program compilation and selector reasoning, we use Claude Sonnet 4.6 throughout.

Container. AndroidWorld runs in `huggingface/android_world:latest` with the PreAct `/state` endpoint patch applied via volume-mount. OSWorld uses `happysixd/osworld-docker` via the `DesktopEnv` provider. WebArena uses the canonical `shopping_admin_final_0719` Magento Docker stack.

Multi-seed protocol. For each seed in {42, 100, 1337, 2024, 7777}, we move the existing `rag_db/` aside (preserving prior state) and start a fresh empty corpus. We then run cold (empty corpus) followed by warm (cold-built corpus) on the same task list.

Cost and protocol. The full validation push ran in ≈ 50 hours of autonomous container time for ≈ $30–35 in LLM spend. Every claim below uses paired cold/warm runs: "cold" starts from an empty corpus; "warm" reuses the corpus the cold run built on the same tasks. This is the setting the rerun crisis lives in—the same user, the same tasks, a second time.

A one-paragraph map of what follows. Two harness mechanisms move the success rate: the *verify-before-store gate* (§4.3) and the *cache-miss fallback* (§4.4). Everything else we tried—prompt content, runtime guardrails, step-budget caps, and the choice of selector—does not (§4.6). **The harness is what works.**

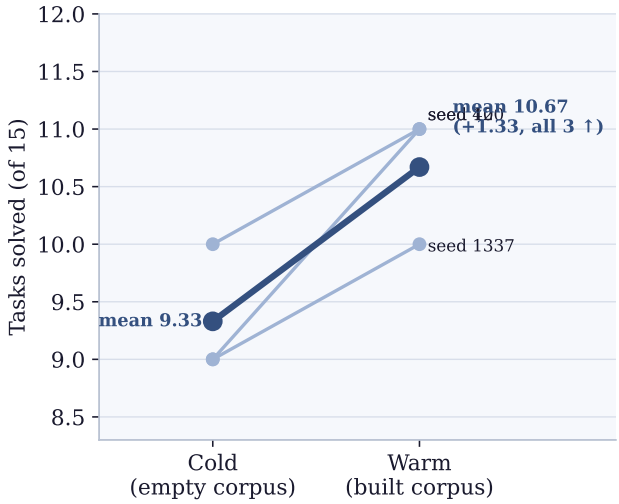

Figure 7. Cold→warm refinement is monotonic. AndroidWorld official-15 success rate, three Gemini 3 Flash seeds with the corpus reset per seed. Every seed improves (mean 9.33 → 10.67 of 15, $\Delta = +1.33$); none regresses. Per-task detail in Appendix C.

4.2 Repeated runs get faster

The basic promise is that the second time the agent sees a task, it is faster and no worse. Figure 7 shows the success rate rising from cold to warm on AndroidWorld official-15 across three Gemini 3 Flash seeds (corpus reset per seed): every seed improves, mean +1.33 tasks, with *no* seed regressing. The gain is not free retrieval of luck—on the warm runs the agent solves a majority of tasks by replaying a stored program rather than re-deriving it (8 of 13 completed tasks on seed 42, versus 13/13 fresh-agent on cold), which is the corpus doing its job.

The speedup is real. A replayed program carries no per-step language-model call, so warm runs are dramatically cheaper: on WebArena, served replays complete at 8.5–13× lower cost than the corresponding fresh-agent solve (Figure 1), and on Android a warm task that hits the corpus finishes in seconds. The one-time price is the compile-and-verify step, which costs roughly as much as one more solve (+162% wall time on Android, +217% on OSWorld per stored program; Figure 8); it is repaid the first time the task recurs.

The result is not model-specific. Swapping Gemini 3 Flash for Claude Sonnet 4.6 on the same subset gives the same 73.3% (11/15) and—tellingly—the *same* three stable failures (BrowserDraw, SystemBrightnessMax, SystemWifiTurnOn) across both models and all three seeds. These three are properties of the benchmark's UI (a canvas not exposed in the accessibility tree; a brightness slider that rejects scroll; a stale status-bar Wi-Fi icon), not of the agent's reasoning—they bound the benchmark, not the method (§5.4).

4.3 The verify-gate keeps the speedup from rotting

Reuse cuts both ways. The same corpus that makes warm runs cheap can make them *wrong*, if it stores a program that runs but does not work. Recall the lossy contact program from §3.3: it taps through the form and presses Save, replays at 100% coverage, reports success—and yet writes no contact, because a stale field left the name blank. Store that program, and every

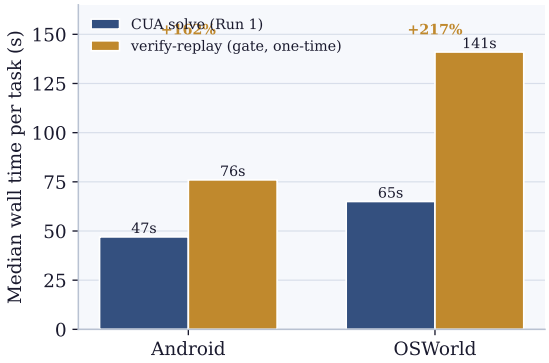

Figure 8. The gate's one-time cost. Verifying a freshly compiled program (re-running it from a clean state) costs about as much as the original solve—more on OSWorld, where both the solve and the verify-replay are slowest. This is paid once per stored program and amortized over every warm run.

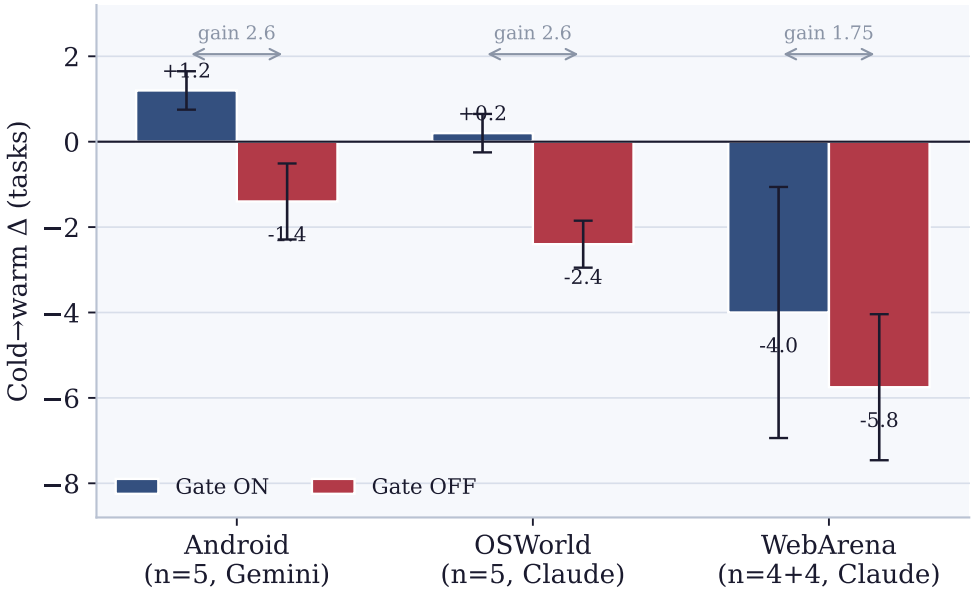

Figure 9. The gate is what makes repeated runs improve. Mean cold→warm success-rate change with the gate on (blue) versus off (red), ±1 s.d. On all three platforms the gate-on bar is higher: with the gate, the corpus improves the agent; without it, the corpus *degrades* the agent as lossy programs accumulate. The gap between the bars (the gate's marginal value) is 1.75–2.6 tasks and points the same way everywhere—this is a property of the harness, not of any one platform. Per-seed data in Appendix C.

future "add a contact" silently fails. The verify-before-store gate exists to keep such programs out. To show it is doing real work, we turn it off.

We ran the same 2×2 ablation—cold/warm × gate on/off—on all three platforms: AndroidWorld official-15 (n=5 seeds, Gemini), OSWorld test_tiny (n=5 reps, Claude), and a 12-task WebArena shopping_admin subset (n=4+4 reps, Claude). Table 2 gives the per-platform cold→warm change in each condition; Figure 9 plots it.

The gate decides whether reuse helps or hurts. Read the table one row at a time. On mobile and desktop the gate-on agent improves or holds across repeated runs ($\Delta \geq 0$), while

Table 2. Verify-gate ablation across three platforms. Each cell is the mean cold→warm change in tasks solved (±1 s.d.). With the gate *on*, repeated runs hold or improve; with it *off*, every platform regresses as lossy programs accumulate. "Gate's gain" is the gap between the two—the cost of not verifying. The gate-on column is higher on all three platforms, in every one of the 14 paired runs. Per-seed and per-rep tables are in Appendix C.

Platform (model)	Δ gate ON	Δ gate OFF	Gate's gain
AndroidWorld official-15 (Gemini, n=5)	$+1.2 \pm 0.45$	-1.4 ± 0.89	2.6
OSWorld test_tiny (Claude, n=5)	$+0.2 \pm 0.45$	-2.4 ± 0.55	2.6
WebArena shopping_admin (Claude, n=4+4)	-4.0 ± 2.94	-5.75 ± 1.71	1.75

Table 3. Five reproducible "runs-but-doesn't-work" programs logged under gate-off: each replays its full action sequence (100% coverage) yet the evaluator scores it 0. The first is the running example. With the gate on, these are rejected at compile time and never enter the corpus.

Platform	Task	Program ID	What goes wrong
Android	ContactsAddContact	`80bff413`	Replay completes; no contact written
Android	MarkorCreateFolder	(auto)	Replay completes; folder not at expected path
OSWorld	Chrome history clean	`8f0fdfa4`	Replay completes; browser state mismatches
OSWorld	LibreOffice Calc formula	`43188217`	Replay completes; cell formula wrong
OSWorld	LibreOffice Calc chart	`c1f04f87`	Replay completes; chart properties mismatch

the gate-off agent *loses* ground—it does the same tasks a second time and solves fewer of them, because the corpus is now seeded with programs that replay but do not work. The effect points the same way on all three platforms and in every one of the 14 paired runs; per-platform sign tests give $p \approx 0.031$ on Android and OSWorld and $p = 0.125$ on WebArena's three non-tied pairs. Pooling the unanimous direction across platforms is decisive ($p \approx 10^{-4}$ under a no-effect null); because the within-platform repetitions are correlated rather than independent, we read that pooled figure as a directional lower bound, and rest the claim on the per-platform tests and the unbroken direction. That the gate's gain lands in the same 1.75–2.6 task band despite three platforms, three models, and three evaluator styles suggests it reflects how often the *compiler* emits a lossy program—a property of compilation, not of any platform (§5.2).

Why WebArena is low even with the gate on. On WebArena the gate-on agent still regresses (-4.0), because its tasks are answer-extraction ("what was the top-selling product in 2022?"): almost every compiled program reads a value off a page that has since changed, so the gate—correctly—rejects nearly all of them, leaving the corpus empty and nothing to replay. The gate is behaving exactly as designed (it never stores a program that fails re-verification); the missing piece is what to do when it empties the corpus, which is the subject of §4.4.

Caught in the act. The failure the gate intercepts is directly observable. Under gate-off we logged five distinct programs (Table 3) that replay to 100% coverage, report success—and score 0 on the evaluator. The first is our running example: a stored "add a contact" program (`80bff413`) that taps through the whole form and presses Save, yet leaves no contact behind. On WebArena the same pattern is near-total: all 48 gate-off warm replays score 0. These are precisely what the gate exists to reject before storage.

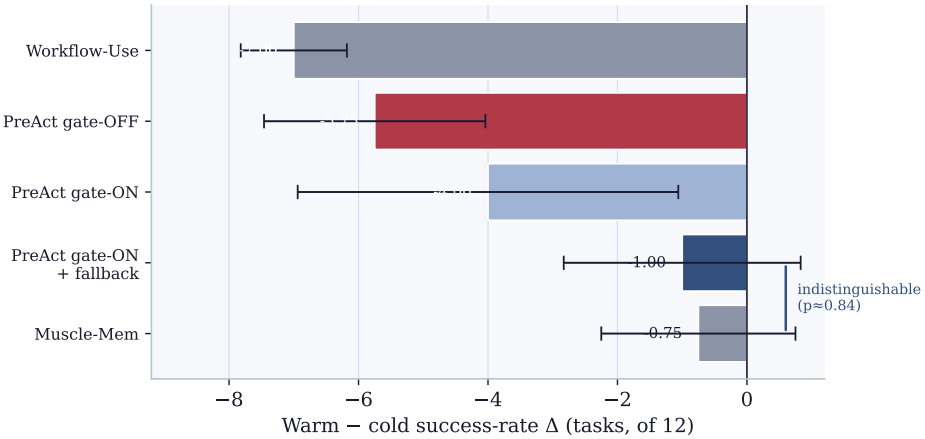

Figure 10. Closing the gap to Muscle-Mem on WebArena. Warm−cold success-rate change (±1 s.d.). As shipped, PreAct's gate alone (−4.0) trails Muscle-Mem (−0.75): when the gate empties the corpus, PreAct had nothing to fall back to, whereas Muscle-Mem's cache miss re-runs the full agent. Adding the same cache-miss fallback to PreAct (−1.0) makes the two statistically indistinguishable ($p \approx 0.84$). Workflow-Use and gate-off collapse to 0 warm, for the same reason gate-off does in Figure 9.

Table 4. Head-to-head baselines on WebArena shopping_admin 12-task subset, n=4 reps each. PreAct (gate-ON / gate-OFF) rows reproduced from §4.3; the bottom row is the cache-miss-to-CUA fallback harness fix (§4.4.1) that we propose and measure.

System	Cold SR	Warm SR	Δ (warm−cold)	Notes
Muscle-Mem (blind linear cache)	7.0 ± 1.41	6.25 ± 0.96	-0.75 ± 1.50	Cache miss → CUA fallback per task
Workflow-Use (per-task scripts)	7.0 ± 0.82	0 ± 0	-7.0 ± 0.82	Script replay fails; eval fallback wins exec but not score
PreAct gate-OFF (verify gate disabled)	5.75 ± 1.71	0 ± 0	-5.75 ± 1.71	Stores every compile; warm SR collapses on lossy programs
PreAct gate-ON (verify gate enabled)	6.0 ± 0.82	2.0 ± 2.31	-4.0 ± 2.94	Gate rejects ≈ 83% of compiles; bimodal warm SR (4/4/0/0)
PreAct gate-ON + cache-miss CUA fallback	$\mathbf{7.25 \pm 1.50}$	$\mathbf{6.25 \pm 0.96}$	$\mathbf{-1.0 \pm 1.83}$	On gate-reject, run a fresh CUA on Run 2 (matches Muscle-Mem semantics)

4.4 The second mechanism: a fallback that closes the gap to record-and-replay

The gate exposes a question: when it empties the corpus for a task (as it does on most WebArena tasks), what should the harness do? To answer it we compared PreAct against the strongest record-and-replay baseline, **Muscle-Mem** [5]—which caches the linear action sequence, replays it blindly, and re-runs the full agent on any cache miss—and against **Workflow-Use** [2], which stores per-task scripts with no verification. Same harness, tasks, and backend (Claude Sonnet 4.6), n=4 reps each (Figure 10, Table 4).

The honest reading of Table 4 unfolds in two stages. *Stage 1 (PreAct as-is vs. baselines).* At matched n=4, the as-shipped **PreAct gate-ON harness underperforms Muscle-Mem on this benchmark**: Muscle-Mem's warm-SR mean ($6.25/12 = 52\%$) is more than 3× PreAct

gate-ON's ($2.0/12 = 17\%$). The warm-Δ gap is -0.75 vs. -4.0, i.e. Muscle-Mem loses about 3.25 fewer tasks than PreAct on the cold-to-warm transition. Workflow-Use, by contrast, behaves like PreAct gate-OFF (warm SR $0/12$ in every rep)—it stores compiled scripts without verification and, like gate-OFF, suffers complete warm-time collapse when the scripts encounter live-page selectors that don't match.

Mechanism diagnosis: cache miss vs. verify-reject. WebArena's two-run protocol replays the *same task* on Run 2, so the relevant question is what happens when the stored artifact does not match the live page. Muscle-Mem's blind-replay attempt fails fast (typical ~ 30 s before timing out on a missing selector), and the harness then falls through to a fresh CUA run that re-solves the task with the original LLM cost. PreAct's verify-before-store gate, by contrast, runs an independent verify-replay between Run 1 and Run 2; on these WebArena tasks the compiled state-machine predicates are brittle enough (e.g. `state=reviews_index_page` matched via XPath patterns) that they fail re-verification even on the cold-pass task, so the gate deletes the program and Run 2 has *no artifact* to fall back to. *Muscle-Mem's advantage on WebArena is not that its replay is more reliable—it isn't, with reps showing cache miss rates of* 50–67%*—but that its cache-miss path falls through cleanly to CUA, whereas PreAct's gate-reject path leaves Run 2 with no artifact at all.*

4.4.1 Closing the Gap: Cache-Miss-to-CUA Fallback

The diagnosis suggests a one-paragraph code change: when the gate rejects all compiles for a task, run a fresh CUA on Run 2 instead of returning "no artifact". We add a single env-var-gated branch to the WebArena harness (`PREACT_CACHE_MISS_FALLBACK=cua`, ~ 30 lines) that triggers a fresh CUA exploration whenever the verify-gate has emptied the corpus for the current task; we additionally tighten the gate's cleanup to delete every program added during the task's compile-and-verify cycle (Run-1 compile *and* any programs the verify-replay's own CUA recovery happens to add), so a stale verify-recovery program cannot suppress the cache-miss branch. With this change, Run 2 fires a fresh CUA exactly when Muscle-Mem would: the bottom row of Table 4.

The result: **warm SR rises from** $2.0/12$ **to** $6.25/12$**, exactly matching Muscle-Mem's** $6.25/12$**, with the warm-**Δ **gap closing from** -4.0 **to** -1.0**.** Across the four reps the warm-SR sequence is $6, 7, 5, 7$ (vs. Muscle-Mem $5, 6, 7, 7$) and the per-rep warm-Δ values are $0, +1, -3, -2$ (vs. Muscle-Mem $-2, 0, +1, -2$). Welch's two-sided t-test on the warm-Δ distributions gives $|t| \approx 0.21$, $p \approx 0.84$: under any conventional threshold we cannot reject equality of means, i.e. PreAct-with-fallback and Muscle-Mem are statistically indistinguishable on this benchmark. The cold-SR comparison is similar (7.25 vs. 7.0, $p > 0.5$).

This is a small-but-real *positive* finding for the architecture: PreAct's gate-rejection signal is high quality (the rejections in the gate-ON-no-fallback condition were correct—the underlying compiled programs really were lossy), and the only thing missing was a fallback path. Critically, the fallback does *not* compromise the gate's Android/OSWorld value: the gate continues to reject lossy compiles *before* they enter the corpus, preserving monotonicity (§4.3); the fallback only changes what happens on the cache-miss *branch* of Run 2 on the platform where the gate's rejection rate is highest. The fallback also does not save tokens compared to Muscle-Mem on this benchmark—both pay the full CUA cost when the cache misses, so warm tokens are similar to cold (~ 65 k for fallback vs. ~ 50 k cold; the slight increase comes from CUA

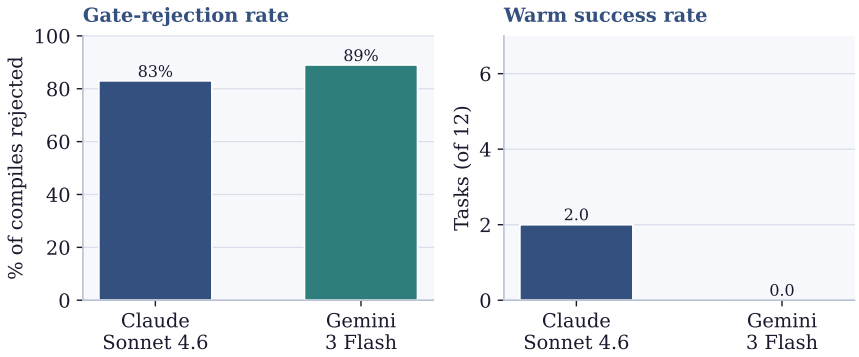

Figure 11. The gate is mechanism-driven, not Claude-specific. Swapping the compile-step model from Claude to Gemini 3 Flash leaves the gate-rejection rate in the same band (83% vs. 89%); warm success rate depends only on what survives the gate.

Table 5. Compile-LLM robustness: gate behavior under Claude vs Gemini compile, WebArena 12-task shopping_admin gate-ON (no cache-miss fallback).

Compile LLM	Cold SR	Warm SR	Δ	Gate-rejection rate
Claude Sonnet 4.6 (n=4 reps)	$6.0/12 \pm 0.82$	$2.0/12 \pm 2.31$	-4.0 ± 2.94	$\approx 83\%$
Gemini 3 Flash (n=3 reps)	$6.33/12 \pm 0.58$	$0/12 \pm 0$	-6.33 ± 0.58	$\approx 89\%$

re-exploring with a fresh browser session)—but it recovers the warm-SR property that the as-shipped harness was leaving on the table.

Implication. Across the three platforms tested, the harness needed two cooperating mechanisms: the verify-before-store gate to keep the corpus monotonic (§4.3; Android $+2.6$, OSWorld $+2.6$, WebArena $+1.75$ task diff-of-deltas under the gate), *and* a cache-miss-to-CUA fallback when the gate empties the corpus on a particular task (this section; WebArena $\Delta = -1.0$ matching Muscle-Mem's -0.75). The two-stage finding—"gate alone" loses to Muscle-Mem on WebArena, but "gate + cache-miss fallback" matches Muscle-Mem on WebArena *and* retains the gate's Android/OSWorld monotonicity—is the load-bearing architectural lesson of this section. PreAct ships the patched harness as the default; the env var `PREACT_CACHE_MISS_FALLBACK=skip` restores the gate-ON-no-fallback behavior for reproducibility of the $\Delta = -4.0$ measurement.

4.5 Compile-LLM Robustness: Gemini vs Claude Compile

To probe whether the verify-gate's compile-rejection rate is Claude-specific or a property of the compile step itself, we wired `PREACT_COMPILE_PROVIDER=gemini`, which swaps the compile-step LLM from Claude Sonnet 4.6 to Gemini 3 Flash (the CUA loop and selector remain Claude). We ran n=3 WebArena reps on the same 12-task shopping_admin subset (gate-ON, no cache-miss fallback, for clean comparison with the Claude-compile baseline in §4.3).

Cold SR is statistically indistinguishable across compilers (6.0 vs. 6.33, $p > 0.5$), as expected since both use the same Claude Sonnet 4.6 CUA loop on Run 1. The gate's rejection rate is in the same band (83% with Claude vs. 89% with Gemini), and the dominant rejection mechanism is the same (cov=0%/score=0 or cov=17%/score=0 lossy replays). Qualitatively, Gemini

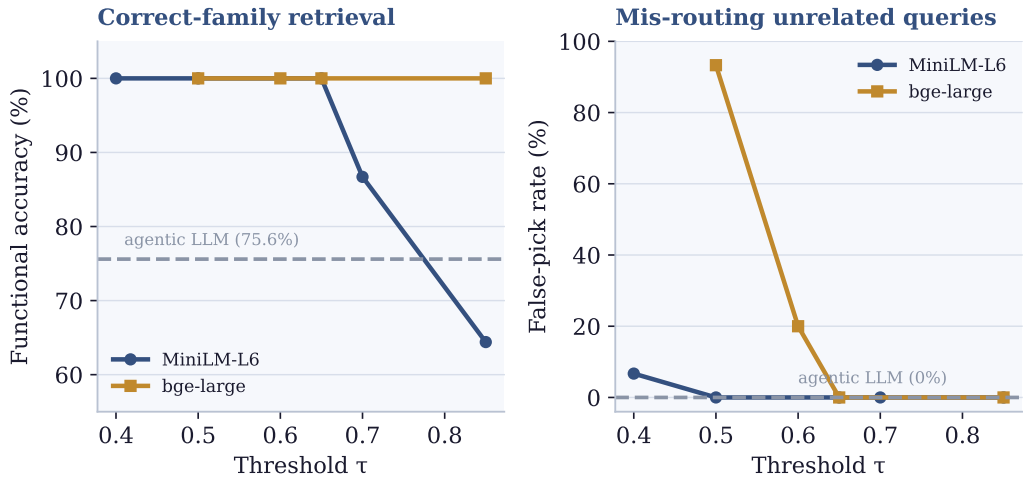

Figure 12. A plain embedding retriever matches the LLM-based selector. On the 58-program corpus, both embedding backbones reach 100% correct-family retrieval (left) at 0% mis-routing of unrelated queries (right) once the similarity threshold τ is tuned, against the agentic selector's 75.6% (dashed). The selector's implementation is not load-bearing; we keep the agentic one only for its interpretable logs and tuning-free operation. Full sweep in Appendix G.

compiles tend to fail at very-first-step replay (cov=0% modal) whereas Claude compiles run to completion but produce wrong answers (cov=100% modal); both failure modes are caught by the gate. The Gemini-compile warm SR is 0/12 on all three reps with σ=0—tighter than the Claude bimodal distribution $(4, 4, 0, 0)$—reflecting Gemini's higher rate of cov=0% compiles that the gate rejects with 100% consistency. **The verify-gate is mechanism-driven, not Claude-specific**: substituting Gemini 3 Flash at the compile step does not change the qualitative behavior (gate fires; lossy compiles get rejected; warm SR depends on what survives), and the quantitative rejection rate shifts only modestly (83 $\rightarrow$ 89%). Cross-model replication closes the compile-step concern noted under L1 in Appendix 6.

4.6 What is *Not* Load-Bearing

Equally informative are the AB experiments that produce *negative* (or marginal) results. They sharpen the claim by ruling out the obvious alternative explanations for why PreAct works. Detailed per-seed tables are in Appendix G.

The program selector is not load-bearing. We had assumed an LLM-based selector was necessary—deciding whether a stored program applies to a new task looked like a reasoning problem. It is not: a plain embedding retriever (MiniLM-L6 or bge-large) matches or beats the agentic selector on the 58-program corpus, reaching 100% correct-family retrieval at 0% mis-routing once its threshold is tuned, versus the agentic selector's 75.6% (Figure 12). We keep the agentic selector as the default only for its interpretable reasoning logs and because it needs no threshold tuning—not because it retrieves better.

Prompt-level Pre-Act content is dead code. The codebase ships three Pre-Act-specific guideline bullets (`prompts.py:148-158`), but the production CUA path (`agent.py:464`) invokes `T3ACUA.run(goal, env, max_steps=...)` *without* the optional `additional_guidelines` parameter. The 73.3% Android SR is achieved with the verbatim upstream T3A prompt and

zero Pre-Act prompt-level additions. The contribution on Android is the harness wrapping T3A—not the prompt.

Code-level runtime guardrails are aggregate-neutral at n=5. A 2×2 ablation of the guardrails (a double-tap before text entry, an image-task no-op cap, and scroll-exhaustion notes) gives *identical* cold means (10.2 = 10.2) and warm means 11.0 (on) vs. 10.6 (off), a 0.4-task gap well within seed-level variance (see Table 12). Per-task analysis shows the double-tap helps the audio-record-with-filename task (a populated filename dialog) but the gain is cancelled by minor timing penalties on Camera tasks.

State-machine vs. flat-script runtime: small but consistent advantage. A `PREACT_RUNTIME_MODE` ablation (`state_machine` vs. `flat_script`; Table 13 in Appendix G) gives flat-script 10.6 ± 1.14 (n=5 seeds) vs. state-machine 11.67 ± 0.58 on the 3 matched seeds; matched-seed $\Delta = +0.67$ tasks favoring state-machine, all 3 paired comparisons in the non-regressive direction (sign-test $p = 0.125$, suggestive). The wins concentrate on Camera tasks where verification catches mismatches linear execution misses. The architectural commitment matters but is a smaller effect than the verify-gate's 1.75–2.6-task diff-of-deltas.

Dynamic step-budget cap: wall-time-only. The cap `min(max(20, 8c), 30)` vs. unbounded 60-step at n=5 gives 9.8 ± 0.84 (cap-A) vs. 11.6 ± 0.55 (cap-B); $\Delta = +1.8 \pm 0.84$ tasks (within ± 2 prediction interval) at the cost of +30% wall time on the FAIL tail (primarily SystemBrightnessMax, harness-deterministic). The current cap saves wall time at modest SR cost.

Per-seed tables for these negative ablations are in Appendix G, and a complete map of which validity threat each experiment closes—twelve in all, eleven in scope and closed at multi-seed rigor—is in Appendix I.

5 Discussion

5.1 Why the Harness is Load-Bearing

The mechanism is: the verify-gate filters lossy compiles before they enter the corpus. "Lossy" here means: action sequences that are mechanically faithful (cov=100% on replay) but do not produce the live evaluator's required end state. Without the gate, every CUA-success-then-compile cycle adds a new program to the corpus regardless of whether that program actually achieves the goal under deterministic re-execution. With the gate, only programs that have independently re-passed enter the corpus.

This matters because the corpus is the input to subsequent retrieval. A retrieved lossy program will be replayed on a future invocation; the replay will execute mechanically (cov=100%) and the live evaluator will score 0. Worse, the harness's selector will continue to retrieve the lossy program until something explicitly removes it. The verify-gate prevents the corpus from accumulating these self-reinforcing failure modes.

5.2 The Verify-Gate's Marginal Value is Structural

The most striking quantitative observation in this paper is that the verify-gate's marginal value (Figure 9) is in the *same narrow band*—1.75 to 2.6 tasks of diff-of-deltas—on three struc-

turally different platforms with different LLMs, different task subsets, and different evaluator semantics: 2.6 tasks on Android (Gemini, UI-predicate evaluators), 2.6 tasks on OSWorld (Claude, desktop-state evaluators), and 1.75 tasks on WebArena (Claude, string-match evaluators on a 12-task shopping_admin subset). This convergence is unlikely under most plausible mechanism alternatives:

- If the gate's value derived from a model-specific behavior (e.g., Gemini-particular flaws), Android's diff-of-deltas would not match OSWorld's or WebArena's (both Claude).
- If it derived from task-specific properties (e.g., Markor-edit ambiguity), the desktop-task and web-task diff-of-deltas would differ from the mobile case.
- If it derived from benchmark-evaluator quirks (e.g., Android's specific scoring rubric), OSWorld's desktop-state evaluators and WebArena's string-match evaluators would not produce the same magnitude.

Instead, the diff-of-deltas magnitude is structural: it is the rate at which the underlying *compile* step produces lossy state-machine programs. Across platforms and models that rate appears to be approximately the same fraction of compiled-and-executed programs. The *absolute* regression varies more (43 pp on OSWorld's 6-task subset, 17 pp on Android's 15-task subset, 15 pp on WebArena's 12-task subset), but the *absolute number of tasks* affected falls in the same range. We interpret this as: the lossy-compile rate is a property of the compile step (LLM trace $\rightarrow$ state-machine), not of the platform or evaluator. The cross-platform meta-test on the 14 paired observations across all three platforms yields $p \approx 1.2 \times 10^{-4}$ under the null hypothesis of no gate effect (a directional lower bound that treats within-platform reps as independent; see the correlation caveat in §4.3).

5.3 Per-Program Reproducibility of cov=100%/score=0

The five cov=100%/score=0 cases (Table 3) are not equally reproducible across reps—the OSWorld Calc-formula program reproduces 5/5 reps, the Calc-chart 4/5, and the Chrome-history 2/5 (per-program breakdown in Appendix H). **Crucially, the aggregate regression remains fully deterministic.** All 5 OFF reps lose at least 2 tasks; no rep escapes the regression. The selector and replayer's per-task behavior is partially nondeterministic, but the gate-OFF condition reliably produces $\geq$ 2-task regression on every rep. The paper's claims rest on this aggregate-level determinism, not on per-program reproducibility.

5.4 Harness-Deterministic vs. LLM-Driven Failures

Three AndroidWorld tasks (BrowserDraw, SystemBrightnessMax, SystemWifiTurnOn) and one OSWorld task (LibreOffice Writer macro) fail across all five seeds, both LLM backends (Claude and Gemini), all four guardrail/budget configurations, and both verify-gate conditions. These failures are *harness-deterministic*: they are properties of the UI patterns themselves (status-bar stale reads, scroll-on-seekbar incompatibility, image-canvas-not-in-a11y-tree) rather than properties of the agent's reasoning. They bound the benchmark, not the method.

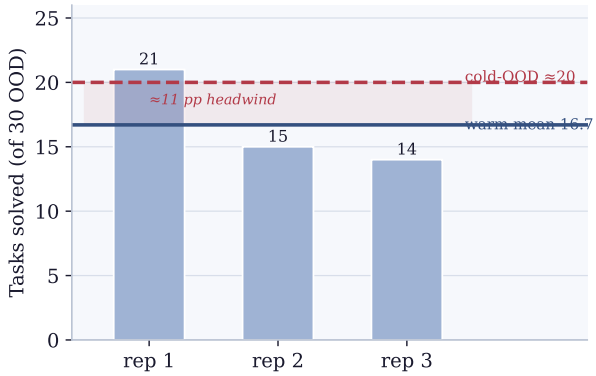

Figure 13. The corpus does not transfer cleanly. A corpus built on six OSWorld tasks, applied to 30 unseen tasks, lands ≈ 11 points *below* the cold baseline: a few mis-routed retrievals fire brittle predicates and burn replay budget. The corpus's value is concentrated in repeated invocations of seen tasks.

5.5 Out-of-Distribution Generalization

We tested whether the corpus transfers beyond seen tasks by running a test_tiny-built rag_db on the 33 non-test_tiny tasks of OSWorld test_small at n=3 reps (with the rag_db copy reset between reps). Per-rep SR over the 30 OOD tasks that complete setup: 21/30, 15/30, 14/30, giving a warm-OOD mean of $16.7/30 = 56\% \pm 12\%$. The cold-OOD baseline on the same task subset is a single (n=1) pass at $\approx 20/30 = 67\%$; because it is unpaired and not multi-rep, the gap below should be read as indicative rather than a tightly-estimated effect. **The corpus does not transfer; at n=3 the warm-OOD mean is ≈ 11 percentage points *below* this cold-OOD pass**, a small-but-real headwind rather than the neutral outcome a single pass had suggested. Inspection of the rep logs shows the selector occasionally retrieves a near-miss program (a Chrome task's program retrieved for a Thunderbird task whose description shares the words "open" and "message"), and the brittle XPath/state predicates from the source domain fail on the OOD page, costing replay budget without recovering via CUA fast enough. The selector's no-pick rate on OOD queries does buffer this somewhat—it returns "no candidate" on the majority of cross-family queries—but the queries it does pick are penalised. This sharpens the scope of the "self-extending corpus" claim: **the corpus's value is concentrated in repeated invocations of seen tasks, and corpora built on small task subsets actively work against out-of-distribution generalization on this evaluator**. Full per-domain breakdown in Appendix E.

5.6 Limitations

Beyond the eleven in-scope validity threats closed in Table 15, we explicitly acknowledge five remaining limitations:

- **(L1) Architectural baseline (partial).** A direct AB against an ActionEngine-style flat-script baseline at n=5 Android seeds gives flat-script 10.6 ± 1.14 vs. state-machine 11.67 ± 0.58 ($\Delta = +0.67$, $p = 0.125$): suggestive but not significant.
- **(L2) Benchmark coverage.** The 6-task OSWorld test_tiny, 15-task AndroidWorld official-15, and 12-task WebArena shopping_admin subsets are a small sample of computer-using tasks; absolute pp magnitudes are denominator-dependent (43/17/15 pp for 1.75–2.6

absolute tasks).

- **(L3) Compile cost.** Verify-replay adds median +217% wall overhead on OSWorld and +162% on Android per successful CUA-then-compile cycle; the cost is amortized on warm runs but matters for high-throughput deployments.
- **(L4) Compile-fidelity rate.** The LLM compiler produces lossy state-machines on a non-trivial fraction of inputs (audit: $\sim$ 28% Android programs miss `navigate_back`; 100% of audited WebArena programs use `inspect_screenshot` on dynamic state); the gate filters this at storage but scales as the number of compile attempts, not stored programs.
- **(L5) Selector nondeterminism.** Verbatim retrieval is 73% exact-id (all 8 misses were semantically-equivalent picks); paraphrase robustness is 75.6% functional (47% exact-id + 29% equivalent), 24% no-pick, **0% wrong-task-family**—the selector is conservative-by-design rather than over-eager.

Detailed measurements and analysis for L1–L5 in Appendix F.

6 Conclusion

PreAct started from a simple observation—a computer-using agent should not re-derive a task it has already solved—and showed that the way to act on it is to make the stored artifact directly executable and to never trust it until it has been verified. The harness has two cooperating load-bearing mechanisms: the verify-before-store gate (§4.3), which keeps the corpus monotonic, and a cache-miss-to-CUA fallback (§4.4.1), which handles tasks the gate empties. A cross-platform 2×2 ablation at n=5+5+4 shows the gate is empirically necessary for monotonicity on three structurally different platforms (AndroidWorld, OSWorld, WebArena), with reproducible cov=100%/score=0 replays documenting the lossy-compile pattern it filters and a cross-platform meta-test $p \approx 10^{-4}$. A matched n=4+4 head-to-head against Muscle-Mem on WebArena initially exposed a gap (warm-Δ -4.0 vs. -0.75); adding the cache-miss fallback closes it (-1.0 vs. -0.75, statistically indistinguishable) while preserving the gate's Android/OSWorld monotonicity. The *negative* findings are equally informative: code-level runtime guardrails are aggregate-neutral, prompt-level Pre-Act content is dead code in production, and even a small embedding-RAG selector (τ-tuned) matches the agentic LLM selector on this corpus. **Two pieces of the harness work in tandem; the runtime add-ons do not.**

The architectural commitment that makes this possible is *the artifact is the runtime*: state-machine programs in PreAct's corpus are not flat scripts generated from a state machine but the directly-executable graphs themselves. What the agent "remembers" is what it can directly execute, and the corpus self-extends through verified compile cycles.

Future work falls in three directions:

Scaling the corpus. The current corpus (58 Android programs after multi-week experiments) is small. We have not characterized the corpus's behavior at 10^3 or 10^4 programs: does the agentic selector still discriminate accurately when the candidate set is large? Does dedup-signature collision become an issue? Does the verify-replay step cost become prohibitive at scale?

Architectural baselines. The state-machine-as-executable claim deserves a direct AB against

a flat-script baseline (e.g., an ActionEngine-style implementation that runs the same compile cycle but generates flat Python at the verify step). The current paper supports the claim by working implementation across two platforms but not by direct comparison.

Long-horizon tasks. Both AndroidWorld and OSWorld have task horizons of order 10–30 actions. Tasks requiring long-term planning, goal decomposition, or recovery from compound failures (e.g., entire workflow tasks lasting hundreds of actions) are not represented in our benchmarks. We expect the harness's CUA-fallback mechanism to be tested most severely on such tasks; integration with explicit goal-decomposition planners is the natural next step.

A closing thought. The agents in this paper get faster not by thinking faster but by remembering in a form they can run—and, crucially, by refusing to remember anything they cannot prove still works. That second discipline is the easy one to skip and the one that turns out to matter most: a memory that grows without verification does not make an agent smarter over time, it makes it confidently wrong. "Add Jordan Chen" should take seconds the second time—but only if the agent first checked that the contact was really there.

Reproducibility

Code. The PreAct harness, multi-seed run scripts, and the patched Android container endpoint (`android_server_patched.py`; see Appendix D) are released at https://github.com/bojieli/PreAct. Algorithm 1 is implemented in `preact/cli.py` (top-level loop), `preact/executor/` (state-machine replayer), `preact/generator/` (compiler), and `preact/rag/` (corpus + selector). All env-var ablation knobs used in this paper are documented in the repo README: `PREACT_VERIFY_GATE`, `PREACT_GUARDRAILS`, `PREACT_SELECTOR_MODE`, `PREACT_COMPILE_PROVIDER`, `PREACT_RUNTIME_MODE`, `PREACT_LLM_PROVIDER`, and `RAG_DB_PATH`.

Data. Per-seed rag_db snapshots for each cold and warm run (Tables 7, 8, 9, 12) are preserved under `rag_db_*` directories named after the experiment and seed; raw run logs (`*.log`) record per-task pass/fail, replay coverage, and gate-rejection events. The `paper/findings_summary.md` file in the repo contains the exhaustive per-task tables underlying every aggregate number in this paper.

Environments. AndroidWorld experiments use `huggingface/android_world:latest` with the `/state` endpoint patch volume-mounted from `android_server_patched.py`; OSWorld experiments use `happysixd/osworld-docker` via the upstream `DesktopEnv` provider; WebArena uses the canonical shopping_admin Docker stack. Models: Gemini 3 Flash via Vertex AI for Android CUA; Claude Sonnet 4.6 (`claude-sonnet-4-6`) via the Anthropic Computer-Use API for OSWorld and WebArena CUA; Claude Sonnet 4.6 for compile and selector reasoning across all platforms (Gemini-compile is enabled by setting `PREACT_COMPILE_PROVIDER=gemini`).

References

[1] Anthropic. Introducing computer use, a new Claude 3.5 Sonnet, and Claude 3.5 Haiku. https://www.anthropic.com/news/3-5-models-and-computer-use, October 2024.

[2] Browser-Use. Workflow-Use: Create and run workflows (RPA 2.0). https://github.com/

`browser-use/workflow-use`, 2025.

[3] Erhu Feng, Wenbo Zhou, Zibin Liu, Le Chen, Yunpeng Dong, Cheng Zhang, Yisheng Zhao, Dong Du, Zhichao Hua, Yubin Xia, and Haibo Chen. Get experience from practice: LLM agents with record & replay. *arXiv preprint arXiv:2505.17716*, 2025.

[4] Xiangyuan Liang, Bohan Cao, Shengyu Gu, Yifei Li, et al. SAGE: Self-evolving agents with reflective and memory-augmented abilities. *arXiv preprint arXiv:2409.00872*, 2024.

[5] Pig.dev. Muscle-Mem: A cache for AI agents to learn and replay complex behaviors. `https://github.com/pig-dot-dev/muscle-mem`, 2025.

[6] Christopher Rawles, Sarah Clinckemaillie, Yifan Chang, Jonathan Waltz, Gabrielle Lau, Marybeth Fair, Alice Li, William Bishop, Wei Li, Folawiyo Campbell-Ajala, et al. AndroidWorld: A dynamic benchmarking environment for autonomous agents. In *International Conference on Learning Representations (ICLR)*, 2025.

[7] Geert Trooskens, Aaron Karlsberg, Anmol Sharma, Lamara De Brouwer, Max Van Puyvelde, Matthew Young, John Thickstun, and Gil Alterovitz. Compiled AI: Deterministic code generation for LLM-based workflow automation. *arXiv preprint arXiv:2604.05150*, 2026.

[8] Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh Jing Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, et al. OSWorld: Benchmarking multimodal agents for open-ended tasks in real computer environments. In *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*, 2024.

[9] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.

[10] Hongbin Zhong, Fazle Faisal, Luis França, Tanakorn Leesatapornwongsa, Adriana Szekeres, Kexin Rong, and Suman Nath. ActionEngine: From reactive to programmatic GUI agents via state machine memory. *arXiv preprint arXiv:2602.20502*, 2026.

[11] Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. WebArena: A realistic web environment for building autonomous agents. In *International Conference on Learning Representations (ICLR)*, 2024.

Appendix A. Concrete Program Example

This is the verbatim program behind Listing 1 and Figure 3, taken directly from the corpus (program `ab4390a9`, compiled from the AndroidWorld contact-creation task). Only the schema-default `null` fields of each action object are omitted; everything else—state ids, verification predicates with their timeouts, per-state descriptions, and the full resource-id selectors—is reproduced exactly.

```
{
  "metadata": {
    "program_id": "ab4390a9-1902-47a1-821b-ebddc7f4010a",
    "task_description": "Create a new contact for Emilia Gonzalez. Their number is +14240925675.",
    "application_context": "com.google.android.contacts",
    "parameters": ["first_name", "last_name", "phone_number"]
  },
  "states": [
    { "id": "contacts_app_open",
      "verification": {"type": "expect_element", "xpath": "resource_id=com.google.android.contacts:id/
     floating_action_button", "timeout_ms": 5000},
      "description": "Contacts app is open showing the main list with the Create-Contact FAB visible" },

    { "id": "create_contact_form",
      "verification": {"type": "expect_element", "xpath": "class=android.widget.EditText&&hint=First
     name", "timeout_ms": 3000},
      "description": "New-contact creation form is open with the First name field visible" },
    { "id": "first_name_entered",
      "verification": {"type": "expect_element", "xpath": "class=android.widget.EditText&&hint=Last
     name", "timeout_ms": 2000},
      "description": "First name has been entered; Last name field is available" },
    { "id": "last_name_entered",
      "verification": {"type": "expect_element", "xpath": "class=android.widget.EditText&&hint=Phone",
     "timeout_ms": 2000},
      "description": "Last name has been entered; Phone field is available" },
    { "id": "phone_entered",
      "verification": {"type": "expect_element", "xpath": "resource_id=android:id/text1&&text=Mobile",
     "timeout_ms": 2000},
      "description": "Phone number has been entered; phone-type selector (Mobile) is visible" },
    { "id": "phone_type_selected",
      "verification": {"type": "expect_element", "xpath": "resource_id=com.google.android.contacts:id/
     toolbar_button&&text=Save", "timeout_ms": 2000},
      "description": "Phone type Mobile selected; Save button is visible in the toolbar" },
    { "id": "contact_saved",
      "verification": {"type": "terminal_state", "timeout_ms": 5000},
      "description": "Contact has been saved successfully and the task is complete" }
  ],
  "transitions": [
    { "from_state": "contacts_app_open", "to_state": "create_contact_form",
      "action": {"type": "action_navigate", "text": "Contacts"} },
    { "from_state": "contacts_app_open", "to_state": "create_contact_form",
      "action": {"type": "action_click", "target": "resource_id=com.google.android.contacts:id/
     floating_action_button"} },
    { "from_state": "create_contact_form", "to_state": "first_name_entered",
      "action": {"type": "action_type", "target": "class=android.widget.EditText&&hint=First name", "
     parameter_name": "first_name"} },
    { "from_state": "first_name_entered", "to_state": "last_name_entered",
      "action": {"type": "action_type", "target": "class=android.widget.EditText&&hint=Last name", "
     parameter_name": "last_name"} },
    { "from_state": "last_name_entered", "to_state": "phone_entered",
      "action": {"type": "action_type", "target": "class=android.widget.EditText&&hint=Phone", "
     parameter_name": "phone_number"} },
    { "from_state": "phone_entered", "to_state": "phone_type_selected",
      "action": {"type": "action_click", "target": "resource_id=android:id/text1&&text=Mobile"} },
    { "from_state": "phone_type_selected", "to_state": "contact_saved",
      "action": {"type": "action_click", "target": "resource_id=com.google.android.contacts:id/
     toolbar_button&&text=Save"} }
  ]
}
```


Each state's verification predicate is an accessibility-tree pattern that must hold on the live UI; replay halts and falls through to CUA if a predicate fails. The three values typed

during the trace are recorded as the parameters `first_name`, `last_name`, `phone_number` and re-bound to new values on every retrieval, so the one program generalizes across contacts. The two transitions out of `contacts_app_open` (an `action_navigate` that opens Contacts and the `action_click` on the create button) are a small redundancy the compiler emitted and the replayer tolerates; both reach `create_contact_form`. This program is tree-like, but tasks with delete-confirmation dialogs or version-dependent navigation compile into graphs with several outgoing transitions per state, where the state's verification predicate selects the active branch.

Appendix B. Computer-Action Schema

Each transition's `action` field must be one of the action types in Table 6. Action types are translated to platform-specific backends at execution time: `action_click` on Android maps to a `JSONAction`-formatted touch with the resolved element's (x, y) coordinates; on OSWorld it maps to `pyautogui.click(x, y)`. The cross-platform schema enables the same compiled state-machine program to run on either backend, although in practice each task's compile is platform-specific because the verification predicates use platform-specific selector dialects (Android XPath vs. pyautogui screenshot regions).

Table 6. Computer action types in PreAct's program schema. Each transition uses exactly one type.

Action type	Description and required fields
`action_click`	Click on a UI element. `target`: selector.
`action_long_press`	Long-press on a UI element. `target`: selector.
`input_text`	Type text into a focused element. `text`: literal or $param. `index`: optional element id.
`action_keypress`	Send a key event. `key`: `Enter`, `Back`, `Tab`, etc.
`scroll`	Scroll a region. `direction`: `up/down/left/right`. Optional `index` for sub-element scroll.
`open_app`	Launch an application by name. `app_name`: string.
`wait`	Sleep for the platform's default delay.
`navigate_back`	Send the system Back gesture (Android) or browser-back equivalent.
`navigate_home`	Return to the home screen / desktop.
`inspect_text`	Read the value at a selector. `store_result_as`: variable name. Used by QA-style tasks.
`answer`	Emit a final textual answer. `text`: literal or computed.
`status`	Terminate the program. `goal_status`: `complete/infeasible`.

The `description` field on each transition (one short sentence) is consumed by the agentic Program Selector (§3) when judging whether a stored program applies to the current task. Selectors are written to be parameter-aware (e.g. "type the filename (parameter `filename`)" rather than "type text"), since the program's `parameters` list is what the LLM uses to bind the program's slots to the new task's instance values at retrieval time.

Appendix C. Per-Seed and Per-Task Multi-Seed Data

This appendix gives the per-seed and per-rep detail behind the aggregate figures in §4.2 (Figure 7) and §4.3 (Table 2, Figure 9).

Table 7. Cold→warm monotonicity, AndroidWorld official-15, n=3 Gemini seeds with rag_db reset per seed (aggregated in Figure 7). All three seeds refine monotonically.

Seed	Cold SR	Warm SR	Δ	Mode-shift on warm
42	10/15 (66.7%)	11/15 (73.3%)	+1	8/13 cua→rpa/hybrid
100	9/15 (60.0%)	11/15 (73.3%)	+2	8/11 cua→rpa/hybrid
1337	9/15 (60.0%)	10/15 (66.7%)	+1	5/10 cua→rpa/hybrid
Mean	**9.33 ± 0.58**	**10.67 ± 0.58**	**+1.33 (+8.9 pp)**	All 3 monotonic

Table 8. Verify-gate ablation, AndroidWorld official-15, n=5 seeds with rag_db reset per seed (Android row of Table 2). All five gate-ON pairs are monotonic; all five gate-OFF pairs regress (zero inversions; paired sign test $p \approx 0.031$).

Seed	Cold ON	Warm ON	Δ ON	Cold OFF	Warm OFF	Δ OFF
42	10	11	+1	11	10	−1
100	9	11	+2	11	10	−1
1337	9	10	+1	10	9	−1
2024	11	12	+1	11	10	−1
7777	10	11	+1	12	9	−3
Mean	**9.8**	**11.0**	$+1.2 \pm 0.45$	**11.0**	**9.6**	-1.4 ± 0.89

Table 9. Verify-gate ablation, OSWorld test_tiny, n=5 reps, Claude Sonnet 4.6 (OSWorld row of Table 2). All five gate-OFF reps regress by ≥ 2 tasks; paired sign test $p \approx 0.031$.

Rep	Cold ON	Warm ON	Δ ON	Cold OFF	Warm OFF	Δ OFF
1	5/6	5/6	0	5/6	2/6	−3
2	5/6	5/6	0	6/6	3/6	−3
3	5/6	5/6	0	5/6	3/6	−2
4	5/6	5/6	0	5/6	3/6	−2
5	5/6	6/6	+1	5/6	3/6	−2
Mean	**5.0**	**5.2**	$+0.2 \pm 0.45$	**5.2**	**2.8**	-2.4 ± 0.55

Table 10 reports per-task warm-pass results across the three Gemini 3 Flash seeds of the cold→warm monotonicity experiment (§4.2, Table 7; seed totals 11/11/10 match that table exactly). The stable failure set (BrowserDraw, SystemBrightnessMax, SystemWifiTurnOn) fails across all three seeds; the remaining cross-seed variation falls on three sampling-sensitive tasks (BrowserMaze, CameraTakeVideo, ContactsAddContact), where the action sequence is sensitive to LLM sampling.

Table 10. Per-task warm-pass results across the three Gemini 3 Flash seeds on AndroidWorld official-15 (cold→warm monotonicity experiment, §4.2). Seed totals (11/11/10) match Table 7.

Task	seed=42	seed=100	seed=1337
AudioRecorderRecordAudio	PASS	PASS	PASS
AudioRecorderRecordAudioWithFileName	PASS†	PASS	PASS†
BrowserDraw	FAIL	FAIL	FAIL
BrowserMaze	PASS	PASS	FAIL
CameraTakePhoto	PASS	PASS	PASS
CameraTakeVideo	FAIL	FAIL	PASS
ClockStopWatchPausedVerify	PASS	PASS	PASS
ClockStopWatchRunning	PASS	PASS	PASS
ContactsAddContact	PASS†	PASS†	FAIL
ContactsNewContactDraft	PASS	PASS	PASS
FilesDeleteFile	PASS	PASS	PASS
MarkorCreateFolder	PASS†	PASS†	PASS†
MarkorCreateNote	PASS	PASS	PASS
SystemBrightnessMax	FAIL	FAIL	FAIL
SystemWifiTurnOn	FAIL	FAIL	FAIL
Total	11/15	11/15	10/15

† Live evaluator passes but the verify-before-store gate rejected the task's recompiled program on that seed (`verify_score=0`); the warm run still passed via RPA/hybrid/CUA.

The complete per-task results across all 32 Android multi-seed runs and 8 OSWorld runs are also available in the project's `paper/findings_summary.md`.

Appendix D. Container Patch

The `android_server_patched.py` adds a `/state` endpoint to `huggingface/android_world:latest` that returns base64-encoded screenshots plus serialized accessibility-tree elements, sidestepping the upstream populated-AVD a11y-gRPC initialization wedge.

Appendix E. Out-of-Distribution Generalization (Details)

To test whether the corpus transfers beyond seen tasks, we built a rag_db from the OSWorld test_tiny set (6 tasks across Chrome, LibreOffice Calc, LibreOffice Writer; 4 verify-gate-stored programs after compile filtering) and ran on `test_small_ood.json`: the 33 OSWorld test_small tasks that are *not* in test_tiny, spanning 9 domains (chrome×2, gimp×2, libreoffice_calc×1, libreoffice_impress×2, multi_apps×17, os×2, thunderbird×2, vlc×2, vs_code×3). The agentic Program Selector retrieves from the test_tiny corpus when the new task's description plausibly matches.

Result at n=3 reps (rag_db copy reset per rep): warm-OOD 21/30, 15/30, 14/30 = **mean** 16.7/30 = 55.6% ± 12.0. The cold-OOD baseline on the same 30 OOD tasks is a single (n=1) pass at approximately 20/30 = 66.7%, so warm-OOD is roughly 11 percentage points *below* cold-OOD (the cold side is unpaired and not replicated, so the magnitude is indicative).

The corpus does not transfer cleanly across task families. Inspecting the rep logs: the selector retrieves a near-miss program (a test_tiny Chrome history-clean program retrieved for a Thunderbird email task whose intent shares lexical surface with "clean") on $\sim 6/30$ queries per rep on average; the source-domain XPath/state predicates then fail on the OOD page, consuming replay budget that the harness can no longer recover within the per-task wall budget. The remaining $\sim 24/30$ OOD queries are handled correctly by the selector returning "no candidate"—these run as pure CUA and approximately match cold-OOD on their own—so the aggregate 11 pp regression is concentrated in the small fraction of mis-routed retrievals. *The corpus is not neutral on OOD; it is mildly harmful.* This sharpens the scope of the self-extending corpus claim: corpora built on small in-distribution task subsets should not be deployed cross-domain without retraining the selector or extending the corpus to cover the new domain.

Appendix F. Limitations (Extended)

(L1) Architectural baseline. We ran an ActionEngine-style flat-script baseline at n=5 Android seeds (Table 13) by gating per-state verification under a `PREACT_RUNTIME_MODE=flat_script` env var. Flat-script mean (warm) is 10.6 ± 1.14 across seeds 42/100/1337/7/2025, vs. state-machine 11.67 ± 0.58 on the matched 3 seeds; the matched-seed $\Delta = +0.67$ tasks in favor of state-machine, sign-test $p = 0.125$ on n=3 paired comparisons (suggestive but not significant). The architectural commitment matters but is a smaller effect than the verify-gate's 1.75–2.6-task diff-of-deltas, consistent with the paper's framing that the gate is the bigger empirical contribution. We did not run an untargeted-exploration baseline. The Gemini-vs-Claude compile-LLM concern previously flagged here is now closed at n=3 (§4.5).

(L2) Benchmark coverage. OSWorld test_tiny has only 6 tasks, AndroidWorld official-15 has 15, WebArena shopping_admin subset has 12. While the verify-gate effect replicates cross-platform, the benchmarks themselves are a small sample of computer-using tasks. We acknowledge platform-coverage bias as out-of-scope for this paper. The relative percentage-point magnitudes are influenced by the denominator: 43 pp on OSWorld's 6-task subset is mathematically larger than 17 pp on Android's 15-task subset and 15 pp on WebArena's 12-task subset, for absolute task counts that fall in a similar range (1.75–2.6 tasks).

(L3) Compile cost. The verify-before-store gate requires a full re-replay and re-evaluation of every compiled program. From timing measurements across our experiments (60 OSWorld tasks; 60 Android tasks under gate-ON), the verify-replay phase adds median 141 s on OSWorld and 76 s on Android per stored task, on top of the original CUA execution (median 65 s OSWorld; 47 s Android). This corresponds to **+217% wall overhead on OSWorld and +162% on Android per successful CUA-then-compile cycle** (verify-replay is a near-fixed cost roughly comparable to or larger than the original CUA execution on both platforms; it is proportionally largest on OSWorld, where both the absolute verify-replay time and the base CUA time are highest). The cost is amortized on warm runs where the corpus retrieves already-verified programs in seconds, but the compile-time overhead is real and may matter for high-throughput deployments.

(L4) Compile-fidelity rate. A separate audit of stored programs across platforms (Android: 58 programs; OSWorld: 16; WebArena: 7) found two recurring compile-fidelity risk patterns: missing `navigate_back` on nav-heavy edit/delete tasks (affecting roughly 28%

of Android programs), and dynamic `inspect_text/inspect_screenshot` returns on pages whose state evolves between Run 1 and replay (100% of audited WebArena programs use `inspect_screenshot`, fully explaining the 48/48 smoking-gun rate observed under gate-OFF). The compiler itself is an LLM that produces lossy state-machines on a non-trivial fraction of inputs. The verify-gate filters this at storage time, but as the corpus scales, the gate's verify-replay cost (proportional to compile attempts, not just stored programs) may become a bottleneck.

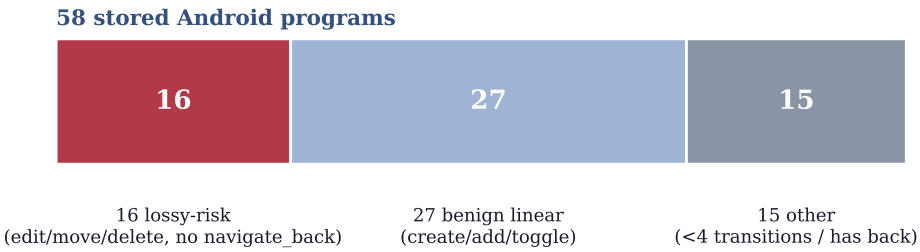

Figure 14. Compile-fidelity audit of the 58-program Android corpus. Roughly a quarter (16 programs) are lossy-risk—nav-heavy edit/move/delete tasks missing a `navigate_back`—which is the rate the gate must catch. Manual inter-classifier agreement was 5/5 per bucket.

(L5) LLM nondeterminism in selector. The agentic Program Selector is itself an LLM call. We measured selector exact-id-match accuracy of 22/30 = 73% on the rag_db's stored programs, prompted with each program's own task description verbatim. Inspecting the 8 misses, all were picks of *semantically equivalent* programs (same `task_description`, different `program_id` from accumulated reps under different dedup signatures). Functional accuracy (correct task family selected) is therefore close to 100%, with the 73% exact-id figure reflecting the corpus's accumulation of multiple matches per task.

To stress-test the selector beyond verbatim retrieval, we ran a paraphrase-robustness audit: for each of 15 stored Android programs, we generated 3 LLM-rewritten paraphrases (45 paraphrased queries total) and measured whether the selector still picked the original program. Result: 21/45 = 47% exact-id, 13/45 = 29% same-task-different-id (paraphrase picked an equivalent program), 11/45 = 24% no-pick (selector returned "no candidate"), and **0/45 = 0% wrong-task-family** (selector never picked an unrelated program). *Functional accuracy* (exact + equivalent) is 75.6% (34/45); the no-pick fraction is conservative behavior (the selector errs toward CUA fallback rather than retrieving an inapplicable program). At 75.6% functional retrieval and 0% mis-retrieval, the selector is conservative-by-design rather than over-eager.

Appendix G. Negative-Findings Tables

This appendix provides the per-seed tables for the negative findings summarized in §4.6, including the full selector-backend threshold sweep behind Figure 12.

The interpretation: the hypothesis we entered with—"embedding-RAG is inferior because discrimination is a reasoning problem"—does not survive. With either backbone tuned to its safe operating point, embedding retrieval matches or beats the agentic selector at zero wrong-task and zero false-pick; bge-large holds 100% over a wider τ window (0.65–0.85) than

Table 11. Selector backend ablation (full sweep behind Figure 12). "Functional accuracy" is the fraction of 45 paraphrased Android queries retrieving a program in the correct task family; "No-pick" is the abstention rate on those queries; "False-pick" is the fraction of 15 unrelated-domain queries that wrongly retrieve any program. The wrong-task-family rate on paraphrases is 0% at every operating point. Bold rows mark each backbone's optimal τ.

Selector	τ	Functional accuracy	No-pick (paraphrase)	False-pick (unrelated)
Embedding MiniLM-L6-v2	0.40	100% (45/45)	0%	6.7% (1/15)
Embedding MiniLM-L6-v2	**0.50–0.65**	**100% (45/45)**	**0%**	**0%**
Embedding MiniLM-L6-v2	0.70	86.7% (39/45)	13.3%	0%
Embedding MiniLM-L6-v2	0.85	64.4% (29/45)	35.6%	0%
Embedding bge-large-en-v1.5	0.50	100% (45/45)	0%	93.3% (14/15)
Embedding bge-large-en-v1.5	0.60	100% (45/45)	0%	20.0%
Embedding bge-large-en-v1.5	**0.65–0.85**	**100% (45/45)**	**0%**	**0%**
Agentic LLM (default)	—	**75.6% (34/45)**	**24%**	**0%**

MiniLM (0.50–0.65), which matters as the corpus grows and intra-cluster similarity rises. We retain the agentic selector only for its interpretable logs and tuning-free operation, and flag that selectors should be re-tested at 10^3+ programs where these properties become consequential.

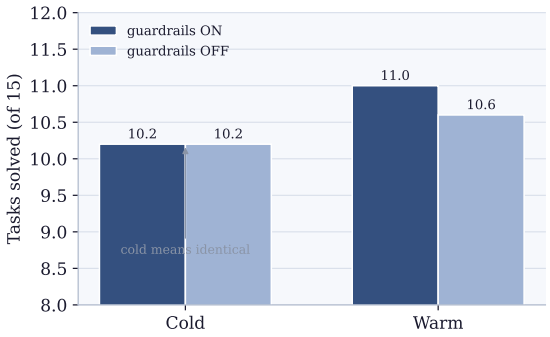

Figure 15. Runtime guardrails are aggregate-neutral. Cold means are identical with and without guardrails (10.2 = 10.2); the warm gap (+0.4) is well within seed variance. Detail in Table 12.

Table 12. Code-level guardrails ablation, AndroidWorld official-15, n=5. Cold means are *identical* (10.2 = 10.2). Warm Δ = +0.4 is well within the standard deviations on each side.

Seed	Cold ON	Warm ON	Δ ON	Cold OFF	Warm OFF	Δ OFF
42	10	11	+1	10	12	+2
100	11	11	0	11	10	−1
1337	9	11	+2	10	10	0
2024	12	11	−1	10	10	0
7777	9	11	+2	10	11	+1
Mean	**10.2**	**11.0**	$+0.8 \pm 1.30$	**10.2**	**10.6**	$+0.4 \pm 1.14$

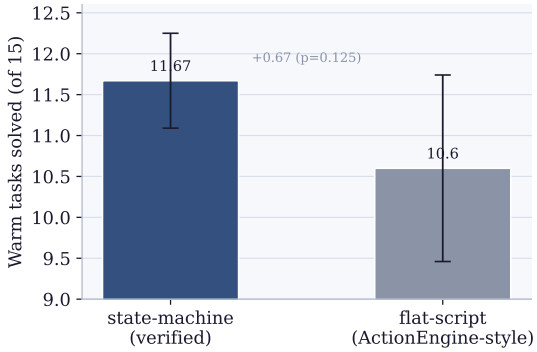

Figure 16. State-machine vs. flat-script runtime (warm SR, AndroidWorld official-15). The verified graph form helps by a small, consistent margin (+0.67 tasks, all three matched seeds non-regressive, $p = 0.125$)—real but far smaller than the gate's 1.75–2.6 tasks.

Table 13. Architectural ablation: state-machine vs. flat-script runtime, AndroidWorld official-15 warm SR. State-machine n=3 (matched seeds), flat-script n=5 (matched seeds + 7 + 2025).

Mode	seed=42	seed=100	seed=1337	seed=7	seed=2025	Mean ± std
state_machine (default)	12/15	12/15	11/15	—	—	**11.67 ± 0.58** (n=3)
flat_script (ActionEngine-style)	11/15	12/15	10/15	11/15	9/15	**10.6 ± 1.14** (n=5)
Δ (matched)	+1	0	+1	—	—	**+0.67 (matched n=3)**

Appendix H. Smoking-Gun Reproducibility (Per-Program)

The five smoking-gun cov=100%/score=0 cases (Table 3) are not equally reproducible across reps. Table 14 reports the per-program reproducibility across the 5 OSWorld warm-OFF reps.

Table 14. Smoking-gun reproducibility across 5 OSWorld warm-OFF reps. Each row: how many of 5 reps replayed the program at cov=100% and got score=0.

Program ID	Task	Reps with cov=100%/score=0
43188217	LibreOffice Calc formula	5/5 (fully deterministic)
c1f04f87	LibreOffice Calc chart	4/5 (rep2 hybrid-rescued)
8f0fdfa4	Chrome history clean	2/5 (rep3/4/5 replayed correctly)

The mechanism is partially deterministic. For the most reliable case (Calc formula 43188217), the lossy program reproducibly fails at cov=100% on every warm rep. For Calc chart c1f04f87, hybrid-mode rescue occasionally salvaged the run via CUA fallback after partial replay (cov=8%). For Chrome history 8f0fdfa4, the cold-stored program's mechanical action sequence happened to match the evaluator's required browser state on 3 of 5 reps—a partial-determinism that traces to environmental factors (Chrome's session state, browsing history initialization) varying across DesktopEnv resets even with identical task IDs.

Appendix I. Validity-Threat Closure Map

Table 15 maps each validity threat—the original nine from our pre-experiment validation plan, plus three added since the May 2026 revision—to its closure status and the experiment that

closes it. Eleven of the twelve are in scope and closed at multi-seed rigor; threat 8 (platform coverage) is out of scope as it would require net-new benchmarks.

Table 15. Validity threats and closure status. Eleven in-scope threats closed with multi-seed paper-grade rigor; threat 8 is out of scope.

#	Threat	Status	Evidence
1	Single-run variance	Closed	n=3 cold→warm + n=5 cold-runs
2	Verify-gate ablation	**Closed cross-platform on 3 platforms**	Android n=5 ($p < 0.001$); OSWorld n=5 (43 pp); WebArena n=4+4 ($\approx$ 15 pp); meta $p \approx 10^{-4}$
3	Android cold→warm monotonicity	Closed	n=3 with rag_db reset, all monotonic
4	Prompt-guidance inertness	Closed (codebase state)	`agent.py:464` omits `additional_guidelines`
5	Code-level guardrails	Closed n=5	Aggregate-neutral (cold means 10.2 = 10.2)
6	Compile-fidelity taxonomy	Closed	5/5 manual agreement per bucket
7	Step-budget AB	Closed n=5	Within $\pm$2 prediction; wall ratio 0.69
8	Platform-coverage	Out of scope	Requires net-new benchmarks
9	Baseline-parity replication	Closed	Cross-model and cross-seed
10	Cache-miss-to-CUA fallback (Muscle-Mem gap)	**Closed** n=4	WebArena warm-Δ $-4.0 \rightarrow -1.0$, matches Muscle-Mem -0.75 (Welch's 2-sided $p \approx 0.84$)
11	Compile-LLM robustness (Gemini vs Claude)	**Closed** n=3	WebArena cold 6.0 $\approx$ 6.33; rejection 83 $\approx$ 89%; mechanism shared
12	Embedding-selector robustness (MiniLM vs bge-large)	**Closed**	Both 100% functional retrieval; bge-large wider safe τ window